



Robotic Systems I

Lecture 10: Robots that Walk

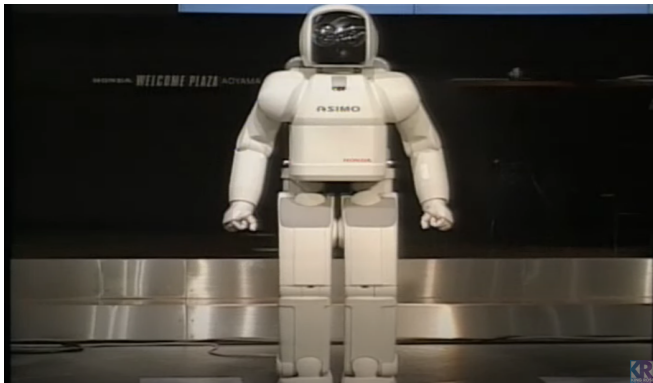
Konstantinos Chatzilygeroudis - costashatz@upatras.gr

Department of Electrical and Computer Engineering
University of Patras

Template made by Panagiotis Papagiannopoulos



Robots that Walk!



Why is Walking Hard?

- **Underactuated:** the robot is not bolted to the world.

Why is Walking Hard?

- **Underactuated:** the robot is not bolted to the world.
- **Hybrid dynamics:** contacts appear and disappear.

Why is Walking Hard?

- **Underactuated:** the robot is not bolted to the world.
- **Hybrid dynamics:** contacts appear and disappear.
- **Impacts:** feet hit the ground with non-smooth dynamics.

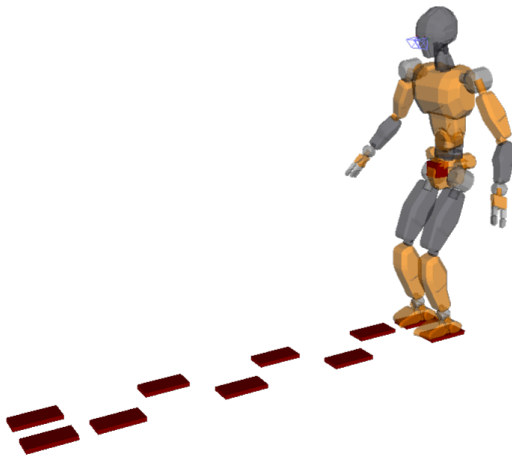
Why is Walking Hard?

- **Underactuated:** the robot is not bolted to the world.
- **Hybrid dynamics:** contacts appear and disappear.
- **Impacts:** feet hit the ground with non-smooth dynamics.
- **Balance:** falling is always nearby.

Why is Walking Hard?

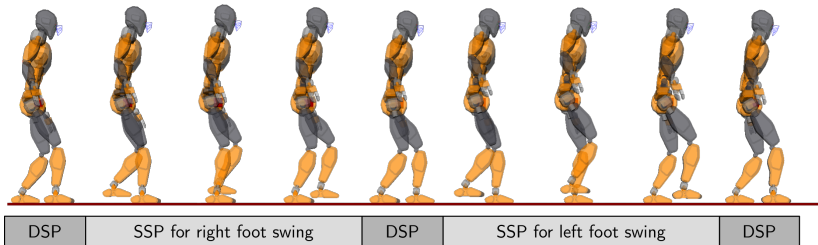
- **Underactuated:** the robot is not bolted to the world.
- **Hybrid dynamics:** contacts appear and disappear.
- **Impacts:** feet hit the ground with non-smooth dynamics.
- **Balance:** falling is always nearby.
- **Uncertainty:** terrain, friction, delays, modeling errors.

Footsteps!



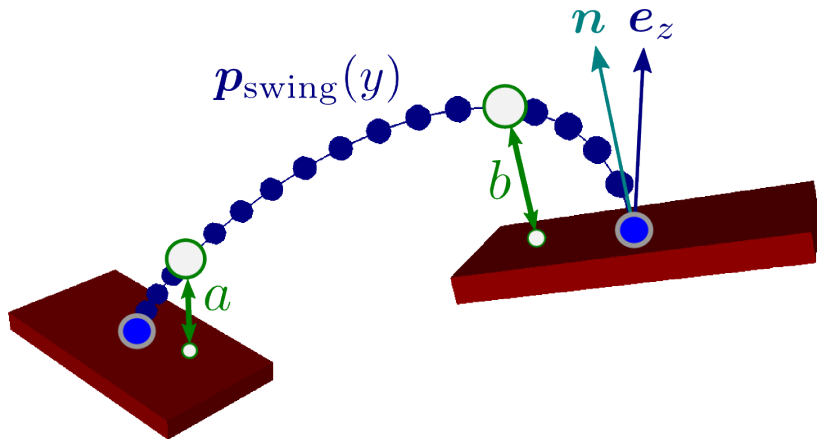
From <https://scaron.info/>

Finite State Machine



From <https://scaron.info/>

Swing Foot Motion



From <https://scaron.info/>

Cubic-Hermite Spline!

- Footsteps define **where** and **when** contacts happen.

Generating Footsteps

- Footsteps define **where** and **when** contacts happen.
- Usually planned in the horizontal plane:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- Footsteps define **where** and **when** contacts happen.
- Usually planned in the horizontal plane:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- Inputs:
 - desired velocity
 - terrain constraints
 - kinematic reachability
 - balance constraints

Simple Footstep Pattern

- For flat terrain, we can start with a nominal gait:

$$x_{i+1} = x_i + v_x T$$

$$y_i = \begin{cases} +w/2 & \text{left foot} \\ -w/2 & \text{right foot} \end{cases}$$

Simple Footstep Pattern

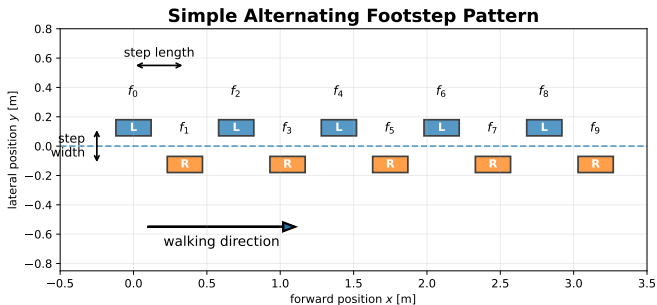
- For flat terrain, we can start with a nominal gait:

$$x_{i+1} = x_i + v_x T$$

$$y_i = \begin{cases} +w/2 & \text{left foot} \\ -w/2 & \text{right foot} \end{cases}$$

- Parameters:
 - step duration T
 - step width w
 - desired forward velocity v_x

Simple Footstep Pattern: Example



But Open-loop Footsteps Are Fragile

- If the robot is pushed, nominal footsteps may fail.

But Open-loop Footsteps Are Fragile

- If the robot is pushed, nominal footsteps may fail.
- The body velocity may differ from the desired velocity.

But Open-loop Footsteps Are Fragile

- If the robot is pushed, nominal footsteps may fail.
- The body velocity may differ from the desired velocity.
- The next footstep should help recover balance.

Foot placement is feedback control.

- Originally used for dynamically balancing legged robots.

Raibert's Foot Placement Heuristic

- Originally used for dynamically balancing legged robots.
- Idea: place the foot based on current body velocity.

Raibert's Foot Placement Heuristic

- Originally used for dynamically balancing legged robots.
- Idea: place the foot based on current body velocity.
- A common form:

$$x_{\text{foot}} = x_{\text{hip}} + \frac{T}{2}\dot{x} + k_v(\dot{x} - \dot{x}_{\text{des}})$$

Interpreting Raibert's Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \underbrace{\frac{T}{2} \dot{x}}_{\text{neutral point}} + \underbrace{k_v(\dot{x} - \dot{x}_{\text{des}})}_{\text{velocity correction}}$$

$\underbrace{\hspace{10em}}_{\text{nominal}}$

Interpreting Raibert's Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \underbrace{\frac{T}{2} \dot{x}}_{\text{neutral point}} + \underbrace{k_v(\dot{x} - \dot{x}_{des})}_{\text{velocity correction}}$$

nominal

- If the robot moves too fast:

$$\dot{x} > \dot{x}_{des}$$

place the foot farther forward.

Interpreting Raibert's Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \underbrace{\underbrace{\frac{T}{2}\dot{x}}_{\text{neutral point}}}_{\text{nominal}} + \underbrace{k_v(\dot{x} - \dot{x}_{des})}_{\text{velocity correction}}$$

- If the robot moves too fast:

$$\dot{x} > \dot{x}_{des}$$

place the foot farther forward.

- If the robot moves too slowly, place it less far forward.

Interpreting Raibert's Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \underbrace{\underbrace{\frac{T}{2}\dot{x}}_{\text{neutral point}}}_{\text{nominal}} + \underbrace{k_v(\dot{x} - \dot{x}_{des})}_{\text{velocity correction}}$$

- If the robot moves too fast:

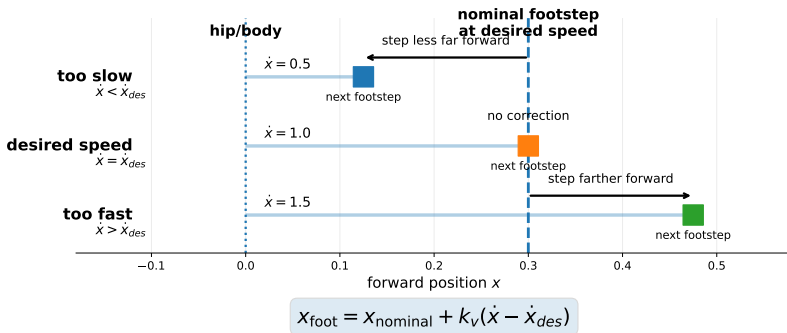
$$\dot{x} > \dot{x}_{des}$$

place the foot farther forward.

- If the robot moves too slowly, place it less far forward.
- This changes the stance dynamics and regulates velocity.

Raibert Heuristic Visualization

Raibert Heuristic: Foot Placement as Velocity Feedback



2D Footstep Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \frac{T}{2} \dot{x} + k_x (\dot{x} - \dot{x}_{\text{des}})$$

$$y_{\text{foot}} = y_{\text{hip}} + s \frac{w}{2} + \frac{T}{2} \dot{y} + k_y (\dot{y} - \dot{y}_{\text{des}})$$

- $s = +1$ for left foot, $s = -1$ for right foot.

2D Footstep Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \frac{T}{2} \dot{x} + k_x (\dot{x} - \dot{x}_{des})$$

$$y_{\text{foot}} = y_{\text{hip}} + s \frac{w}{2} + \frac{T}{2} \dot{y} + k_y (\dot{y} - \dot{y}_{des})$$

- $s = +1$ for left foot, $s = -1$ for right foot.
- w is nominal step width.

2D Footstep Heuristic

$$x_{\text{foot}} = x_{\text{hip}} + \frac{T}{2}\dot{x} + k_x(\dot{x} - \dot{x}_{des})$$
$$y_{\text{foot}} = y_{\text{hip}} + s\frac{w}{2} + \frac{T}{2}\dot{y} + k_y(\dot{y} - \dot{y}_{des})$$

- $s = +1$ for left foot, $s = -1$ for right foot.
- w is nominal step width.
- k_x, k_y tune feedback strength.

- The heuristic gives a desired footstep, but it must be feasible.

Footstep Constraints

- The heuristic gives a desired footstep, but it must be feasible.
- Kinematic limits:

$$x_{\min} \leq x_{\text{foot}} - x_{\text{hip}} \leq x_{\max}$$

$$y_{\min} \leq y_{\text{foot}} - y_{\text{hip}} \leq y_{\max}$$

- The heuristic gives a desired footstep, but it must be feasible.
- Kinematic limits:

$$x_{\min} \leq x_{\text{foot}} - x_{\text{hip}} \leq x_{\max}$$

$$y_{\min} \leq y_{\text{foot}} - y_{\text{hip}} \leq y_{\max}$$

- Terrain limits: avoid obstacles, edges, holes.

- The heuristic gives a desired footstep, but it must be feasible.
- Kinematic limits:

$$x_{\min} \leq x_{\text{foot}} - x_{\text{hip}} \leq x_{\max}$$

$$y_{\min} \leq y_{\text{foot}} - y_{\text{hip}} \leq y_{\max}$$

- Terrain limits: avoid obstacles, edges, holes.
- Dynamic limits: contact forces and friction must be feasible.

- For flat feet, a footprint is not just a point:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- For flat feet, a footprint is not just a point:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- (x_i, y_i) defines the foot center.

- For flat feet, a footprint is not just a point:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- (x_i, y_i) defines the foot center.
- θ_i defines the foot yaw orientation.

- For flat feet, a footprint is not just a point:

$$f_i = (x_i, y_i, \theta_i, t_i)$$

- (x_i, y_i) defines the foot center.
- θ_i defines the foot yaw orientation.
- The whole sole defines a support polygon:

$$\mathcal{P}_i(\theta_i)$$

A flat footprint is a pose, not a point.

Choosing Footstep Orientation

- A simple rule is to align the foot with the desired walking direction:

$$\theta_i = \theta_{\text{body},i}^{\text{des}}$$

Choosing Footstep Orientation

- A simple rule is to align the foot with the desired walking direction:

$$\theta_i = \theta_{\text{body},i}^{\text{des}}$$

- For turning, interpolate the yaw:

$$\theta_{i+1} = \theta_i + \text{clip}(\Delta\theta_{\text{des}}, -\Delta\theta_{\text{max}}, \Delta\theta_{\text{max}})$$

Choosing Footstep Orientation

- A simple rule is to align the foot with the desired walking direction:

$$\theta_i = \theta_{\text{body},i}^{\text{des}}$$

- For turning, interpolate the yaw:

$$\theta_{i+1} = \theta_i + \text{clip}(\Delta\theta_{\text{des}}, -\Delta\theta_{\text{max}}, \Delta\theta_{\text{max}})$$

- Then place the next foot in the local body frame:

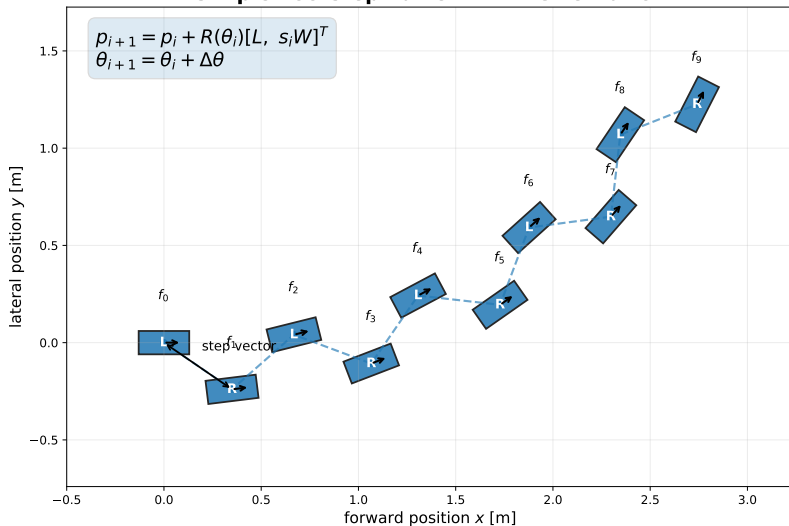
$$p_{i+1} = p_i + R(\theta_i) \begin{bmatrix} L \\ s_i W \end{bmatrix}$$

$$s_i = \begin{cases} +1 & \text{left foot} \\ -1 & \text{right foot} \end{cases}$$

Step length L , step width W , max turn per step $\Delta\theta_{\text{max}}$.

Footstep with Orientation: Example

Simple Footstep Pattern with Orientation



Footstep Generation Pipeline

- 1 Start from desired velocity.

Footstep Generation Pipeline

- 1 Start from desired velocity.
- 2 Generate nominal alternating footsteps.

Footstep Generation Pipeline

- 1 Start from desired velocity.
- 2 Generate nominal alternating footsteps.
- 3 Add feedback using body velocity.

Footstep Generation Pipeline

- 1 Start from desired velocity.
- 2 Generate nominal alternating footsteps.
- 3 Add feedback using body velocity.
- 4 Project the step into the reachable region.

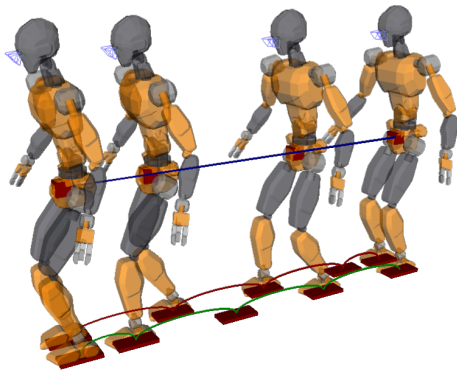
Footstep Generation Pipeline

- 1 Start from desired velocity.
- 2 Generate nominal alternating footsteps.
- 3 Add feedback using body velocity.
- 4 Project the step into the reachable region.
- 5 Check terrain and contact feasibility.

Footstep Generation Pipeline

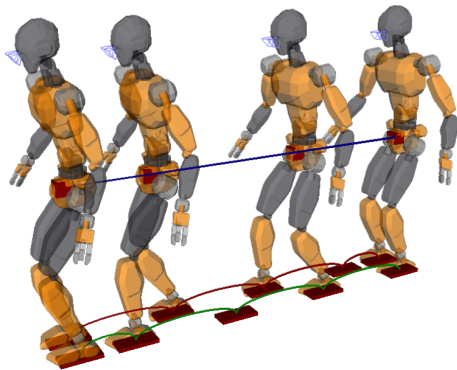
- 1 Start from desired velocity.
- 2 Generate nominal alternating footsteps.
- 3 Add feedback using body velocity.
- 4 Project the step into the reachable region.
- 5 Check terrain and contact feasibility.
- 6 Send footsteps to the CoM / MPC planner.

Whole Body IK Interpolation



From <https://scaron.info/>

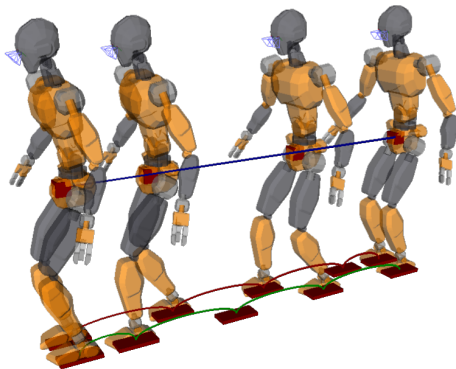
Whole Body IK Interpolation



From <https://scaron.info/>

- **Not dynamically feasible!!**

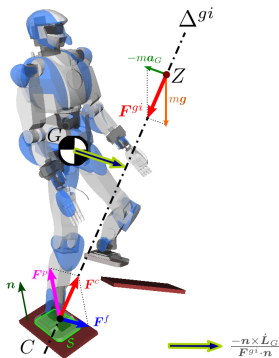
Whole Body IK Interpolation



From <https://scaron.info/>

- Not dynamically feasible!!
- What else can we do?

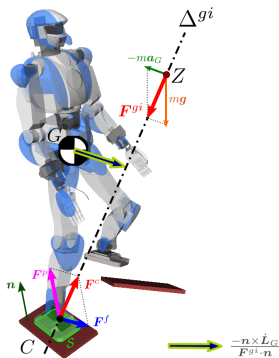
Static Balancing



From <https://scaron.info/>

- CoM should stay in the support polygon!

Static Balancing



From <https://scaron.info/>

- CoM should stay in the support polygon!
- Or rather, the Zero-tilting Moment Point should be in the support polygon!

- A robot is **statically balanced** if gravity does not create a tipping moment.

- A robot is **statically balanced** if gravity does not create a tipping moment.
- For slow motions, this means:

$$p_{\text{CoM}} \in \mathcal{P}_{\text{support}}$$

- A robot is **statically balanced** if gravity does not create a tipping moment.
- For slow motions, this means:

$$p_{\text{CoM}} \in \mathcal{P}_{\text{support}}$$

- $\mathcal{P}_{\text{support}}$ is the convex hull of all contact points.

- A robot is **statically balanced** if gravity does not create a tipping moment.
- For slow motions, this means:

$$p_{\text{CoM}} \in \mathcal{P}_{\text{support}}$$

- $\mathcal{P}_{\text{support}}$ is the convex hull of all contact points.
- If the CoM projection leaves the support polygon, the robot tips over.

Static walking: move the CoM safely from one support polygon to the next.

IK can produce motion that **looks like walking**.

Dynamics asks whether the motion can actually be **executed without falling**.

IK can produce motion that **looks like walking**.

Dynamics asks whether the motion can actually be **executed without falling**.

- Are the contact forces feasible?

IK can produce motion that **looks like walking**.

Dynamics asks whether the motion can actually be **executed without falling**.

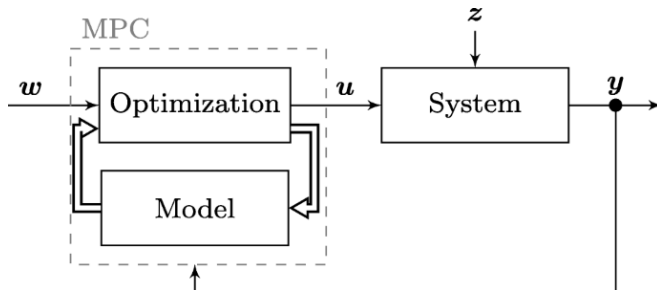
- Are the contact forces feasible?
- Is the ZMP inside the support polygon?

IK can produce motion that **looks like walking**.

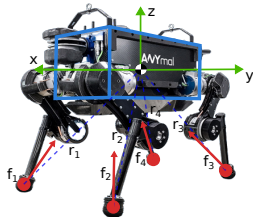
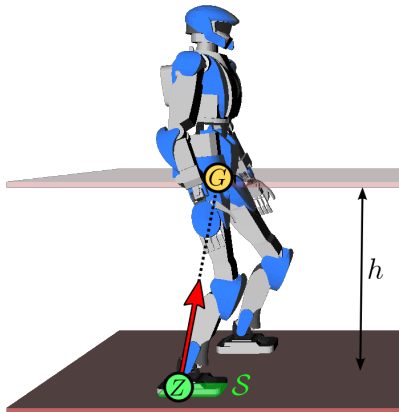
Dynamics asks whether the motion can actually be **executed without falling**.

- Are the contact forces feasible?
- Is the ZMP inside the support polygon?
- Can the robot recover from disturbances?

Model Predictive Control



Simplified Models



But Walking is Dynamic

- During walking, the robot accelerates.

But Walking is Dynamic

- During walking, the robot accelerates.
- Then balance is not determined only by the CoM position.

But Walking is Dynamic

- During walking, the robot accelerates.
- Then balance is not determined only by the CoM position.
- The important quantity becomes the **Zero Moment Point**:

$$p_{ZMP}$$

But Walking is Dynamic

- During walking, the robot accelerates.
- Then balance is not determined only by the CoM position.
- The important quantity becomes the **Zero Moment Point**:

$$p_{\text{ZMP}}$$

- The ZMP is the point on the ground where the net horizontal tipping moment is zero.

Dynamic balance: keep the ZMP inside the support polygon.

- If

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{support}},$$

the contact forces can create a stabilizing moment.

- If

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{support}},$$

the contact forces can create a stabilizing moment.

- If the ZMP reaches the edge of the foot, the foot starts to rotate.

- If

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{support}},$$

the contact forces can create a stabilizing moment.

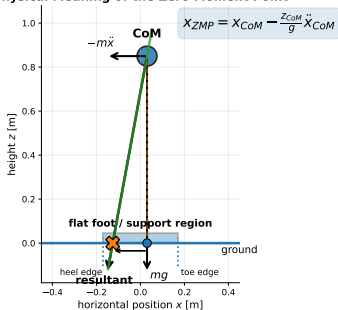
- If the ZMP reaches the edge of the foot, the foot starts to rotate.
- If it leaves the support polygon, the planned motion is not physically feasible.

$$p_{\text{ZMP}} = p_{\text{CoM}} - \frac{z_{\text{CoM}}}{g} \ddot{p}_{\text{CoM}}$$

Linear inverted pendulum model, constant CoM height.

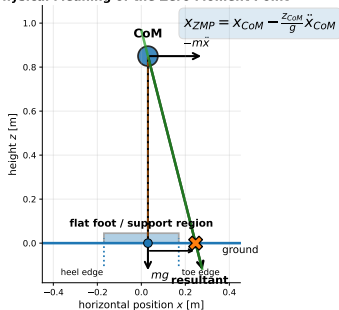
ZMP Visualization

Physical Meaning of the Zero Moment Point



ZMP inside the foot: feasible

Physical Meaning of the Zero Moment Point



ZMP outside the foot: tipping risk

Planning a ZMP Trajectory

- Once footsteps are known, we know the future support polygons.

Planning a ZMP Trajectory

- Once footsteps are known, we know the future support polygons.
- Choose a desired ZMP trajectory inside them:

$$p_{\text{ZMP}}^{\text{des}}(t) \in \mathcal{P}_{\text{support}}(t)$$

Planning a ZMP Trajectory

- Once footsteps are known, we know the future support polygons.
- Choose a desired ZMP trajectory inside them:

$$p_{\text{ZMP}}^{\text{des}}(t) \in \mathcal{P}_{\text{support}}(t)$$

- During double support, the ZMP can move from one foot to the next.

Planning a ZMP Trajectory

- Once footsteps are known, we know the future support polygons.
- Choose a desired ZMP trajectory inside them:

$$p_{\text{ZMP}}^{\text{des}}(t) \in \mathcal{P}_{\text{support}}(t)$$

- During double support, the ZMP can move from one foot to the next.
- During single support, the ZMP must stay inside the stance foot.

Footsteps define **where ZMP is allowed to be.**

- Using the linear inverted pendulum model:

$$p_{ZMP} = p_{CoM} - \frac{z_{CoM}}{g} \ddot{p}_{CoM}$$

From ZMP to CoM Motion

- Using the linear inverted pendulum model:

$$p_{ZMP} = p_{CoM} - \frac{z_{CoM}}{g} \ddot{p}_{CoM}$$

- Given a desired ZMP, we solve for a CoM trajectory.

From ZMP to CoM Motion

- Using the linear inverted pendulum model:

$$p_{ZMP} = p_{CoM} - \frac{z_{CoM}}{g} \ddot{p}_{CoM}$$

- Given a desired ZMP, we solve for a CoM trajectory.
- The CoM does not have to stay directly above the foot.

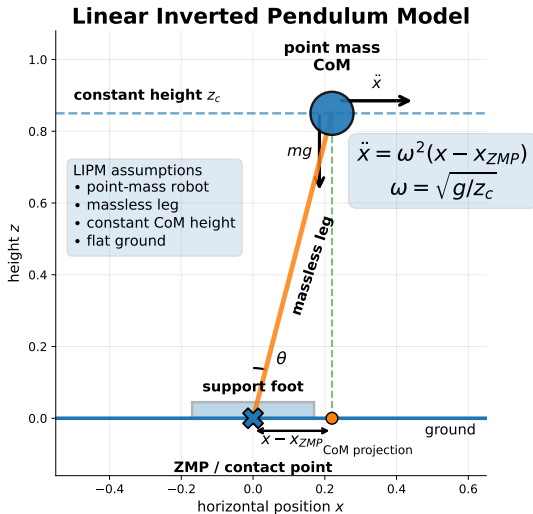
- Using the linear inverted pendulum model:

$$p_{ZMP} = p_{CoM} - \frac{z_{CoM}}{g} \ddot{p}_{CoM}$$

- Given a desired ZMP, we solve for a CoM trajectory.
- The CoM does not have to stay directly above the foot.
- It can move dynamically, as long as the resulting ZMP remains feasible.

CoM planning = choose accelerations that produce a feasible ZMP.

LIPM Visualization



CoM/ZMP Planning Problem

$$\begin{aligned} &\text{find } p_{\text{CoM}}(t) \\ &\text{s.t. } p_{\text{ZMP}}(t) = p_{\text{CoM}}(t) - \frac{z_{\text{CoM}}}{g} \ddot{p}_{\text{CoM}}(t) \\ &\quad p_{\text{ZMP}}(t) \in \mathcal{P}_{\text{support}}(t) \end{aligned}$$

CoM/ZMP Planning Problem

find $p_{\text{CoM}}(t)$

$$\text{s.t. } p_{\text{ZMP}}(t) = p_{\text{CoM}}(t) - \frac{z_{\text{CoM}}}{g} \ddot{p}_{\text{CoM}}(t)$$

$$p_{\text{ZMP}}(t) \in \mathcal{P}_{\text{support}}(t)$$

- This can be solved offline and online!

CoM/ZMP Planning Problem

find $p_{\text{CoM}}(t)$

$$\text{s.t. } p_{\text{ZMP}}(t) = p_{\text{CoM}}(t) - \frac{z_{\text{CoM}}}{g} \ddot{p}_{\text{CoM}}(t)$$

$$p_{\text{ZMP}}(t) \in \mathcal{P}_{\text{support}}(t)$$

- This can be solved offline and online!
- The output is a dynamically feasible CoM trajectory.

Divergent Component of Motion

- The LIPM dynamics are:

$$\ddot{p}_{\text{CoM}} = \omega^2(p_{\text{CoM}} - p_{\text{ZMP}})$$

Divergent Component of Motion

- The LIPM dynamics are:

$$\ddot{p}_{\text{CoM}} = \omega^2(p_{\text{CoM}} - p_{\text{ZMP}})$$

- Define the DCM:

$$\xi = p_{\text{CoM}} + \frac{\dot{p}_{\text{CoM}}}{\omega}$$

Divergent Component of Motion

- The LIPM dynamics are:

$$\ddot{p}_{\text{CoM}} = \omega^2(p_{\text{CoM}} - p_{\text{ZMP}})$$

- Define the DCM:

$$\xi = p_{\text{CoM}} + \frac{\dot{p}_{\text{CoM}}}{\omega}$$

- Then the dynamics split into:

$$\dot{\xi} = \omega(\xi - p_{\text{ZMP}}) \quad (\text{unstable})$$

$$\dot{p}_{\text{CoM}} = \omega(\xi - p_{\text{CoM}}) \quad (\text{stable})$$

DCM is the unstable part of the walking motion.

Why is DCM Useful?

- The ZMP controls the DCM:

$$\dot{\xi} = \omega(\xi - p_{\text{ZMP}})$$

Why is DCM Useful?

- The ZMP controls the DCM:

$$\dot{\xi} = \omega(\xi - p_{\text{ZMP}})$$

- The CoM follows the DCM:

$$\dot{p}_{\text{CoM}} = \omega(\xi - p_{\text{CoM}})$$

Why is DCM Useful?

- The ZMP controls the DCM:

$$\dot{\xi} = \omega(\xi - p_{\text{ZMP}})$$

- The CoM follows the DCM:

$$\dot{p}_{\text{CoM}} = \omega(\xi - p_{\text{CoM}})$$

- To get a bounded walking motion, compute $\xi(t)$ backwards from a final condition.

Why is DCM Useful?

- The ZMP controls the DCM:

$$\dot{\xi} = \omega(\xi - p_{\text{ZMP}})$$

- The CoM follows the DCM:

$$\dot{p}_{\text{CoM}} = \omega(\xi - p_{\text{CoM}})$$

- To get a bounded walking motion, compute $\xi(t)$ backwards from a final condition.
- Then integrate $p_{\text{CoM}}(t)$ forward.

Plan the unstable part, then let the CoM follow it.

Long-Horizon DCM Planning

- Assume future footsteps are known.

Long-Horizon DCM Planning

- Assume future footsteps are known.
- Then the future support polygons are known:

$$\mathcal{P}_{\text{support},k}$$

Long-Horizon DCM Planning

- Assume future footsteps are known.
- Then the future support polygons are known:

$$\mathcal{P}_{\text{support},k}$$

- We plan over a horizon of N timesteps:

$$k = 0, \dots, N$$

Long-Horizon DCM Planning

- Assume future footsteps are known.
- Then the future support polygons are known:

$$\mathcal{P}_{\text{support},k}$$

- We plan over a horizon of N timesteps:

$$k = 0, \dots, N$$

- Decision variables:

$$\xi_k, \quad p_{\text{ZMP},k}$$

Plan the unstable DCM while keeping the ZMP feasible.

- Continuous-time DCM dynamics:

$$\dot{\xi} = \omega (\xi - p_{\text{ZMP}})$$

- Continuous-time DCM dynamics:

$$\dot{\xi} = \omega (\xi - p_{\text{ZMP}})$$

- If p_{ZMP} is constant over one timestep:

$$\xi_{k+1} = a\xi_k + (1 - a)p_{\text{ZMP},k}$$

- Continuous-time DCM dynamics:

$$\dot{\xi} = \omega (\xi - p_{\text{ZMP}})$$

- If p_{ZMP} is constant over one timestep:

$$\xi_{k+1} = a\xi_k + (1 - a)p_{\text{ZMP},k}$$

- where

$$a = e^{\omega\Delta t}, \quad \omega = \sqrt{\frac{g}{z_{\text{CoM}}}}$$

This is linear in ξ_k and $p_{\text{ZMP},k}$.

ZMP Feasibility Constraints

- The ZMP must stay inside the current support polygon:

$$p_{\text{ZMP},k} \in \mathcal{P}_{\text{support},k}$$

- The ZMP must stay inside the current support polygon:

$$p_{\text{ZMP},k} \in \mathcal{P}_{\text{support},k}$$

- For a flat foot or double-support polygon:

$$\mathcal{P}_{\text{support},k} = \{p \mid A_k p \leq b_k\}$$

- The ZMP must stay inside the current support polygon:

$$p_{\text{ZMP},k} \in \mathcal{P}_{\text{support},k}$$

- For a flat foot or double-support polygon:

$$\mathcal{P}_{\text{support},k} = \{p \mid A_k p \leq b_k\}$$

- Therefore:

$$A_k p_{\text{ZMP},k} \leq b_k$$

Known footsteps give known linear constraints.

$$\begin{aligned}
 \min_{\{\xi_k\}, \{p_{\text{ZMP},k}\}} \quad & \sum_{k=0}^{N-1} \|\xi_k - \xi_k^{\text{ref}}\|_{Q_\xi}^2 + \sum_{k=0}^{N-1} \|p_{\text{ZMP},k} - p_{\text{ZMP},k}^{\text{ref}}\|_{Q_z}^2 \\
 & + \sum_{k=0}^{N-2} \|p_{\text{ZMP},k+1} - p_{\text{ZMP},k}\|_{R_z}^2 + \|\xi_N - \xi_N^{\text{ref}}\|_{Q_N}^2 \\
 \text{s.t.} \quad & \xi_{k+1} = a\xi_k + (1-a)p_{\text{ZMP},k}, \quad k = 0, \dots, N-1 \\
 & A_k p_{\text{ZMP},k} \leq b_k, \quad k = 0, \dots, N-1 \\
 & \xi_0 = \xi_{\text{meas}}.
 \end{aligned}$$

Recovering the CoM Trajectory

- Once the DCM trajectory is planned, recover the CoM using:

$$\dot{p}_{\text{CoM}} = \omega (\xi - p_{\text{CoM}})$$

Recovering the CoM Trajectory

- Once the DCM trajectory is planned, recover the CoM using:

$$\dot{p}_{\text{CoM}} = \omega (\xi - p_{\text{CoM}})$$

- In discrete time:

$$p_{\text{CoM},k+1} = \beta p_{\text{CoM},k} + (1 - \beta)\xi_k$$

Recovering the CoM Trajectory

- Once the DCM trajectory is planned, recover the CoM using:

$$\dot{p}_{\text{CoM}} = \omega (\xi - p_{\text{CoM}})$$

- In discrete time:

$$p_{\text{CoM},k+1} = \beta p_{\text{CoM},k} + (1 - \beta)\xi_k$$

- where

$$\beta = e^{-\omega \Delta t}$$

Optimize ξ and p_{ZMP} , then reconstruct p_{CoM} .

Can We Optimize Footstep Locations in a QP?

- Assume the contact sequence and timing are fixed.

Can We Optimize Footstep Locations in a QP?

- Assume the contact sequence and timing are fixed.
- If foot orientations are fixed, footstep positions can be decision variables.

Can We Optimize Footstep Locations in a QP?

- Assume the contact sequence and timing are fixed.
- If foot orientations are fixed, footstep positions can be decision variables.
- The ZMP constraint remains linear:

$$R_i^\top (p_{\text{ZMP},k} - p_i) \in \mathcal{P}_{\text{foot}}^{\text{local}}$$

Can We Optimize Footstep Locations in a QP?

- Assume the contact sequence and timing are fixed.
- If foot orientations are fixed, footstep positions can be decision variables.
- The ZMP constraint remains linear:

$$R_i^\top (p_{\text{ZMP},k} - p_i) \in \mathcal{P}_{\text{foot}}^{\text{local}}$$

- Since R_i is known, this is affine in $p_{\text{ZMP},k}$ and p_i .

Fixed yaw $\Rightarrow$ QP possible.

What If Foot Orientation Is Also Optimized?

- Now the support polygon depends on θ_i :

$$R(\theta_i)^\top (p_{\text{ZMP},k} - p_i) \in \mathcal{P}_{\text{foot}}^{\text{local}}$$

What If Foot Orientation Is Also Optimized?

- Now the support polygon depends on θ_i :

$$R(\theta_i)^\top (p_{\text{ZMP},k} - p_i) \in \mathcal{P}_{\text{foot}}^{\text{local}}$$

- But

$$R(\theta_i) = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix}$$

What If Foot Orientation Is Also Optimized?

- Now the support polygon depends on θ_i :

$$R(\theta_i)^\top (p_{\text{ZMP},k} - p_i) \in \mathcal{P}_{\text{foot}}^{\text{local}}$$

- But

$$R(\theta_i) = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix}$$

- Therefore the constraints are nonlinear in the decision variables.

Variable yaw $\Rightarrow$ not a standard QP.

- **Fix orientations:** solve one QP for CoM/ZMP and foot positions.

- **Fix orientations:** solve one QP for CoM/ZMP and foot positions.
- **Sequential QP:** linearize around nominal foot poses, solve QP, update, repeat.

- **Fix orientations:** solve one QP for CoM/ZMP and foot positions.
- **Sequential QP:** linearize around nominal foot poses, solve QP, update, repeat.
- **Discrete orientations:** choose from candidate yaw angles using mixed-integer optimization.

- **Fix orientations:** solve one QP for CoM/ZMP and foot positions.
- **Sequential QP:** linearize around nominal foot poses, solve QP, update, repeat.
- **Discrete orientations:** choose from candidate yaw angles using mixed-integer optimization.
- **Nonlinear optimization:** optimize foot positions, orientations, and CoM/ZMP together.

Contact timing fixed helps, but variable foot yaw still makes the problem nonlinear.

Can Point Feet Balance?

- With flat feet, the ZMP can move inside the sole:

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{foot}}$$

Can Point Feet Balance?

- With flat feet, the ZMP can move inside the sole:

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{foot}}$$

- With point feet, the support polygon collapses to a point:

$$p_{\text{ZMP}} = p_{\text{foot}}$$

Can Point Feet Balance?

- With flat feet, the ZMP can move inside the sole:

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{foot}}$$

- With point feet, the support polygon collapses to a point:

$$p_{\text{ZMP}} = p_{\text{foot}}$$

- Therefore static single-support balance would require:

$$p_{\text{CoM}} = p_{\text{foot}}$$

Can Point Feet Balance?

- With flat feet, the ZMP can move inside the sole:

$$p_{\text{ZMP}} \in \mathcal{P}_{\text{foot}}$$

- With point feet, the support polygon collapses to a point:

$$p_{\text{ZMP}} = p_{\text{foot}}$$

- Therefore static single-support balance would require:

$$p_{\text{CoM}} = p_{\text{foot}}$$

- Normal point-foot walking is therefore dynamically balanced.

Point-foot bipeds do not stand; they continuously catch themselves.

Point Feet: Balance by Stepping

- Since the ZMP cannot move inside a foot, balance authority comes from the next contact.

Point Feet: Balance by Stepping

- Since the ZMP cannot move inside a foot, balance authority comes from the next contact.
- The DCM / capture point tells us where the robot is falling:

$$\xi = p_{\text{CoM}} + \frac{1}{\omega} \dot{p}_{\text{CoM}}$$

Point Feet: Balance by Stepping

- Since the ZMP cannot move inside a foot, balance authority comes from the next contact.
- The DCM / capture point tells us where the robot is falling:

$$\xi = p_{\text{CoM}} + \frac{1}{\omega} \dot{p}_{\text{CoM}}$$

- A simple idea:

$$p_{\text{next}} \approx \xi$$

Point Feet: Balance by Stepping

- Since the ZMP cannot move inside a foot, balance authority comes from the next contact.
- The DCM / capture point tells us where the robot is falling:

$$\xi = p_{\text{CoM}} + \frac{1}{\omega} \dot{p}_{\text{CoM}}$$

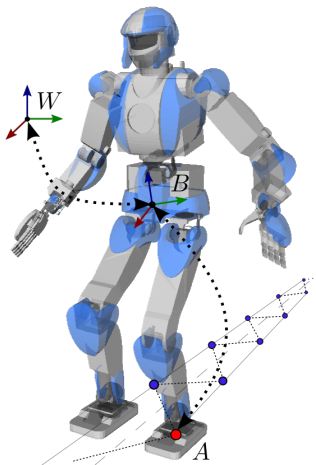
- A simple idea:

$$p_{\text{next}} \approx \xi$$

- If the next foot lands near the capture point, the robot can stop or redirect the fall.

With point feet, foot placement is the main balance input.

Floating-base Estimation!



From <https://scaron.info/>

- 1 Choose footsteps.

Walking Pipeline

- 1 Choose footsteps.
- 2 Define gait phases.

Walking Pipeline

- 1 Choose footsteps.
- 2 Define gait phases.
- 3 Generate swing-foot motion.

Walking Pipeline

- 1 Choose footsteps.
- 2 Define gait phases.
- 3 Generate swing-foot motion.
- 4 Plan CoM / ZMP trajectory.

Walking Pipeline

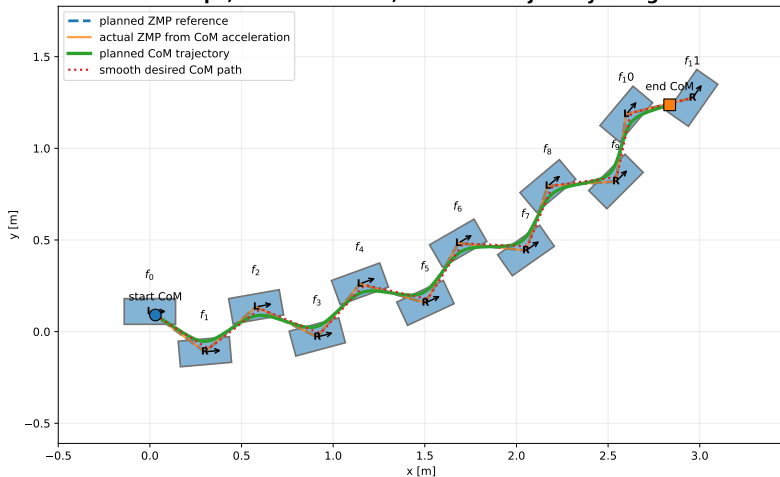
- 1 Choose footsteps.
- 2 Define gait phases.
- 3 Generate swing-foot motion.
- 4 Plan CoM / ZMP trajectory.
- 5 Track with whole-body control.

- 1 Choose footsteps.
- 2 Define gait phases.
- 3 Generate swing-foot motion.
- 4 Plan CoM / ZMP trajectory.
- 5 Track with whole-body control.
- 6 Estimate floating-base state from sensors and contacts.

Walking = planning + balance + feedback.

Planning Example

Footsteps, ZMP Reference, and CoM Trajectory using LIPM



Blog post on walking controllers

Thank you

- **Any Questions?**
- **Office Hours:**
 - **Mon-Tue** (09:00-11:00)
 - 24/7 by email (costashatz@upatras.gr, subject: *ECE_RSI_AM*)
- **Material and Announcements**



Laboratory of Automation & Robotics